



CSI247: Data Structures

Lecture 02c – Review: Java Programming (Arrays)

Department of Computer Science
B. Gopolang (247-275)

Previous Lesson ...

- Java defined classes
 - Scanner, FileReader, PrintWriter, String, etc
 - Their purpose & various methods
 - Need to **import** relevant packages
- User Defined classes
 - Instance variables (make them **private**)
 - Methods
 - Constructors, void, value-returning
 - Parameter passing

- Class implementation
- Testing class methods
 - Tester class
 - Instantiation of class objects (**new** keyword)
 - Invocation of class methods (**dot** operator)
- Representing a class using a UML class diagram
 - Implementing a class given a UML class diagram
- Method signature and overloading

Today ...

Learning Objectives

- By the end of this lecture, you must be able to:
 - Declare and initialize a one-dimensional array
 - 2-D array
 - Perform these basic array operations
 - Add/insert elements
 - Retrieve elements
 - Use arrays as formal parameters to methods
 - Use arrays of Objects
 - e.g. array of Strings, array of Dogs, Accounts

Consider this scenario ...

Student Ab was given this Q:

- Write a Java application that reads 10 lab marks from the keyboard then output all marks that are greater or equal to the average mark.

Ab's attempt →

```
import java.util.*;
public class ComputeAverage {
    public static void main (String[ ] args) {
        Scanner scan = new Scanner (System.in);
        double mark, average, sum = 0.0;
        for (int i = 1; i <= 10; i++) {
            System.out.printf("Enter lab mark");
            mark = scan.nextDouble();
            sum += mark;
        }
        average = sum / 10;
        // left blank
    }
}
```

- Will this approach yield the required answer?
 - No!
 - Ab cannot determine marks $\geq$ average mark
 - Why?
 - Ab will lose all marks except the last one
 - Old mark is ALWAYS replaced by a new mark!
- Possible solution?
 - Keep/store the marks as a **collection**

Collection:

Basically, a group/set of related things (objects).

Any two objects in a collection should be comparable.

- Examples
 - A collection of books.
 - A collection of marks.
 - A collection of Strings, Students, Bank Accounts, etc.
- We can write Java programs that can **create** and **manipulate collections**.

1. Arrays

- **Array:** A collection of items of the same data type!
 - A foundational component in DSs
 - Each item/object is stored in its own cell.
 - Cells are numbered from 0 to `array.length - 1`.
 - Arrays support the following operations:
 - Add/insert an element
 - Retrieve an element
 - Common types of arrays
 - 1D & 2D

Advantages of arrays

- Direct access to data
 - Quick retrieval
- A versatile DS
 - Basic data, e.g. int, char, etc
 - Complex data, e.g. objects
- Storage efficiency
 - Elements occupy contiguous memory locations.

Disadvantages of arrays

- Fixed Size
- Problems with insertion/deletion
- Possible memory allocation problems
- Not flexible enough for complex structures

2. Creating Arrays

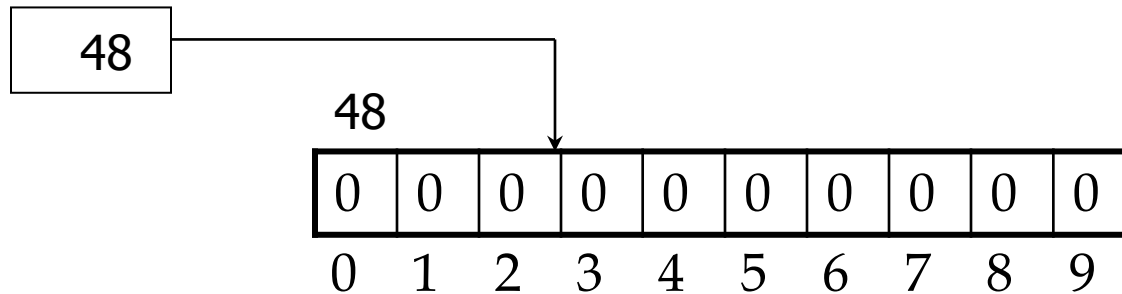
Syntax:

- `dataType[] arr1 = new dataType[intExp];` // 1D
 - intExp: an expression that evaluates to an integral value (byte, short, int, long)

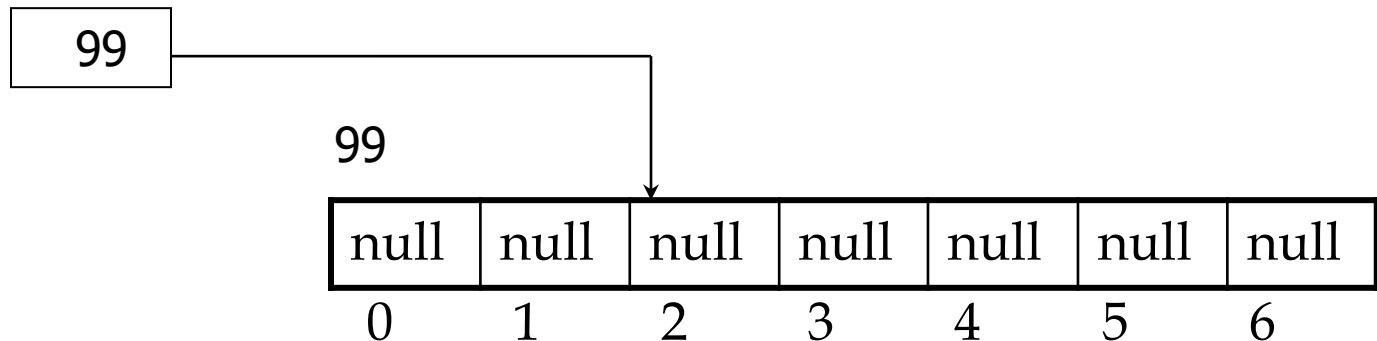
`dataType[][] arr2 = new dataType[intExp1][intExp2];` // 2D

3. Creating Arrays: Memory Model

- `int[ ] numbers = new int[10];` // to hold 10 integer values
numbers



- `String[ ] courses = new String[7];` // to hold 7 references
to String objects
courses



4. Creating and Initializing Arrays

- Declare and Initialize:

```
int[] nums = {10, 6, -7, 9, 19};
```

```
char[] grades = {'A', 'B', 'C', 'D', 'E', 'F'};
```

```
String[] villages = {"Mochudi", "Serowe", "Kanye"}
```

Exercise: Draw memory diagram for the above statements

- Declare, Create, Assign:

```
int[] data;           // declare an array of int values
```

```
data = new int[5]; // create array with space for 5 int values
```

```
data[0] = 10; data[1] = 6; data[2] = -7; ... // store values
```

5. Adding values to an Array

- An array object has a **public attribute `length`**, **not a method**
 - Keeps the number of cells in the array.
- Syntax (for adding a value):
arrayName[intExp] = expression;
 - intExp is an integral expression, called the **index**, satisfying $0 \leq \text{intExp} \leq \text{arrayName.length} - 1$.

Notice that `arrayName[intExp]` is used as a variable!

Example

```
int n = 10;  
int[] numbers = new int[n+4];  
for (int i = 0; i < numbers.length; i++) {  
    if ( i%2 == 1) {  
        numbers[i] = i+1;  
        System.out.printf("%d\n", numbers[i]);  
    }  
}  
System.out.printf("%d\n", numbers.length);
```

Exercise

- a) Draw memory diagram for the above code
- b) What is the output?

6. Array Index Out of Bounds

Array IndexOutOfBounds:

An **error** you get if you try to access a non-existent cell of the array.

- Can only be caught at **run-time**.
- Example

```
int[] numbers = new int[10];
```

- In the above example, array indices go from 0 to `numbers.length - 1`, which is 9.
- These will cause **run-time error**
 - `numbers[-1]` or `numbers[10]` or `numbers[12]`

7. Retrieving Values From An Array

- Rem:
 - Each element in the array occupies its own space
 - To access a value, use its index
- General form used:
 - *arrayName[intExp]*
 - intExp: AKA the index.

● Exercise

- Dry run the following code to determine the contents of array numbers after executing the code segment.

```
int[] numbers = new int[6];  
numbers[0] = 5 ;  
for (int i = 1 ; i < numbers.length; i++) {  
    numbers[i] = i * i + 5;  
    if (i > 2) {  
        numbers[i] = 2 * numbers[i] - numbers[i-1];  
    }  
}
```

8. Arrays as Formal Parameters

- Sometimes we need to pass an array to a method.
- Example

```
public class Mist {  
    public boolean search (int[] listA, int x) {  
        boolean found = false;  
        for (int i = 0; i < listA.length; i++) {  
            if (listA[i] == x )  
                found = true;  
        }  
        return found;  
    }  
}
```

- Invoking the method:

```
int[] numbers = {3,1,-2,6,8};  
int num = 16;  
  
Mist pa = new Mist();  
System.out.printf("%s\n",  
    pa.search(numbers, num));
```

a) Initializing an Array to a Specific Value

```
public void initArray (int[] numbers, int initVal) {  
    for (int i = 0; i < numbers.length; i++) {  
        numbers[i] = initVal;  
    }  
}
```

- Method invoked by giving it an (integer) array and an integer value to initialize array elements with.
 - Array **numbers** MUST have been created, otherwise **NullPointerException** will be raised.

b) Reading Data into an Array

```
public void readIntoArray(int[ ] numbers, Scanner sc)
    for (int i = 0; i < numbers.length; i++) {
        numbers[i] = sc.nextInt();
    }
}
```

- Method invoked by giving it an array of integers followed by a Scanner object.
 - Both the array and the Scanner object MUST have been created and initialized.

c) Printing Array contents

```
public void printArray (int[ ] numbers) {  
    System.out.printf("{"); // for formatting output  
    for (int i = 0; i < numbers.length; i++) {  
        System.out.printf("%d ", numbers[i])  
    }  
    System.out.printf("}");  
}
```

- Method invoked by giving it an integer array
 - The array MUST have been created.

d) Determining Index of the Largest Element

```
public int indexOfMax(int[ ] numbers) {  
    int index = 0;           // Assume numbers[0] has largest element  
    for (int i = 1; i < numbers.length; i++) {  
        if (numbers[i] > numbers[index])  
            index = i;  
    }  
    return index;  
}
```

- Method invoked by giving it (passing) an array of integers.
 - The array MUST have been created.
- Returns position of the largest element in the array.

e) Determining the Average

```
public double computeArrayAverage( int[ ] numbers) {  
    double sum = 0.0, average = 0.0;  
    for (int i=0; i < numbers.length; i++){  
        sum = sum + numbers[i];  
    }  
    if (numbers.length != 0)  
        average = sum / numbers.length;  
    return average;  
}
```

- Method invoked by giving it an (integer) array.
 - The array MUST have been created.
- Returns average of elements in the array.

9. Arrays of String Objects

- **Declaring:**

```
String[] classList = new String[5];
```

- Adding names to the array:

```
classList[0] = "Ame";
```

```
// same as: classList[0] = new String("Ame");
```

- Thereafter, classList[i] can be treated like any String variable:

```
System.out.printf("%d\n", classList[0].length());
```

```
System.out.printf("%s\n", classList[0].toUpperCase());
```

```
System.out.printf("%s\n", classList[0].charAt(2));
```

```
System.out.printf("%d\n", classList[0].indexOf('e'));
```

```
System.out.printf("%d\n", classList[0].indexOf('f'));
```

```
System.out.printf("%s\n", classList[0].equals("ame"));
```

10. Array of Objects of Other Classes

- **Declaring:**

ClassName[] identifier = new ClassName[intExp];

- **Example**

Account[] accs = new Account[5]; // Assuming we have class Account

- To add data to the array:

accs[0] = new Account(190, 450.0);

- *accs[i]* is treated like any Account variable:

- *accs[0].setBalance(500.55);*

- *System.out.printf("%d\n", accs[0].getBalance());*

- *System.out.printf("%s\n", accs[0].equals(accs[3]));*

Summary

1. Collections
2. One Dimensional Arrays
3. Array Index Out of Bounds Error
4. Arrays as Formal Parameters for methods
5. Processing One Dimensional Arrays with Methods
6. Arrays of Objects

Next Lesson ...

- Recursion

Q & A



Thank you